

Acoustic Tracer

Authors

GitHub Repository Link: [Acoustic Tracer](#)

Team Members:

- Alex Wright - Simulation Core / Front-end
- Patryk Mrozek - Simulation Core
- Eoghan Murphy - Simulation Core / Optimization
- Michael McCarthy - Front-end

Introduction

Our project ‘Acoustic Tracer’ is a three-dimensional, acoustic visualiser that allows users to visualise sound travelling throughout a modelled environment as a heat map. At the core it is a C library, where the user can read in a `.glb` file (3D Model File), insert (a) speaker(s), specify simulation settings, and receive back a heat map of how the sound travelled through the environment over time. Our final project extends it into a web-based application that renders this heat map and makes the configuration of the scene and simulation more user-friendly, while also allowing more technical users to achieve their desired configuration. A user can create an account, upload `.glb` model files, view the models in a 3D scene-viewer, configure the scene and simulation settings, and finally run the simulation, returning a heat map which they can play through and replay at a later time if desired. The aim of this project was to create a unique software that could be used by architects, acoustic specialists, or anyone who desires to model how sound travels through their environment. This was also a passion project to explore the potential and concept of ray-tracing, along with creating a software that people would actually use.

Previous Works

On carrying out research into existing technologies before starting this project, we weren’t able to find an exact software to meet our needs.

Architecture

Upon starting the project our first task was to set up the shared GitHub repository where our code was to be hosted. This was an involved process with the whole team, as we wanted to ensure a proper workflow with professional version control standards. We set up an organisation, as to not have a single person hosting the repository. We then created the repository `AcousticTracer` where we each forked it, giving ourselves a personal ‘copy’ of the repository to serve as our `origin`. We then added a remote `upstream` where we could merge changes from our personal forks to the original repository located in the organisation. This allowed us to each work independently on features in parallel and then push these changes into a shared repository. We configured approval rules to ensure that members could not approve their own pull requests, and that each pull request required at least one approval before merging into the main branch.

Once the GitHub had been configured the next step was to work on the C library specification, since that is the core essence of the project. The C Library will henceforth be referred to as the ‘core’ of the project. The members working on the core of the project began at the ‘top’ of the core, creating the specification and outline of the public API presented by the core to the end user. Our initial scope of the project included a ‘command buffer’ that the output would be written to. The idea behind the command buffer was that the output of the simulation would be instructions for any graphics engine (OpenGL, Vulkan, etc.), on how to draw our heat map. This was in the hopes to create a truly independent, graphics library agnostic library, but this was later scrapped due to the complexity. We settled on creating a

consistent communication standard instead, that could be parsed by any front-end, or visualisation tool. This communication standard will be discussed at a later stage.

The architecture / workflow we envisioned for the user was as follows:

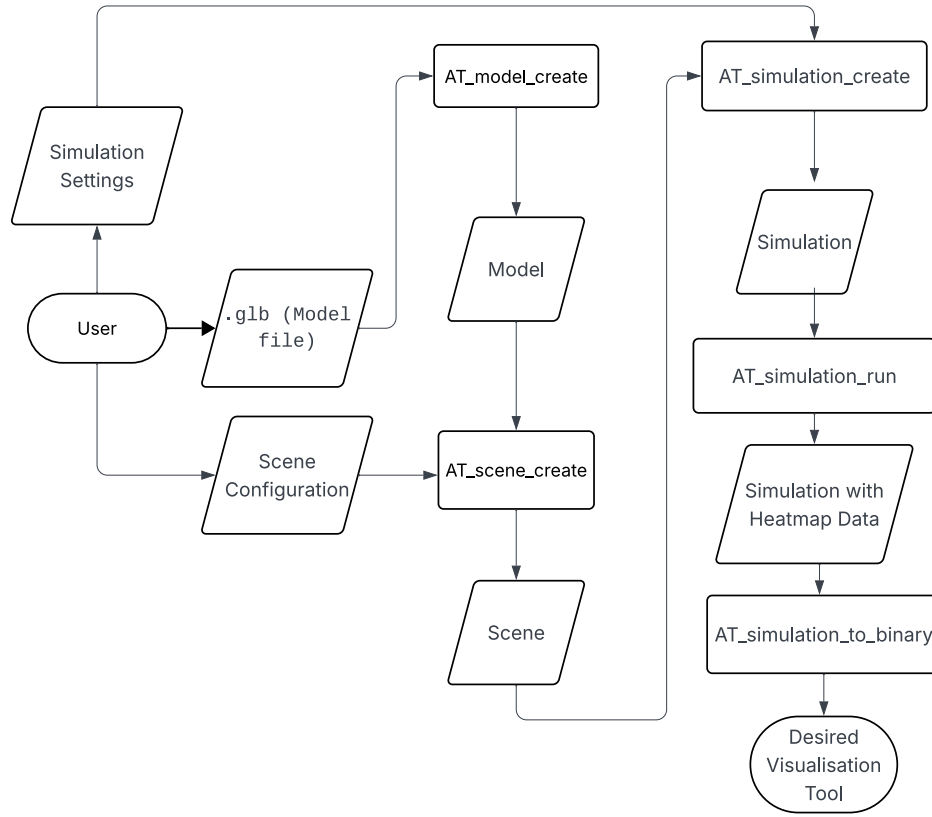


Figure 1: Flowchart of final architecture

The core of the project was written in C, over other languages, for a few key reasons. Firstly, C is extremely performant, which is crucial for our application since it is extremely computationally heavy. A key feature for us was the library **raylib** which C provides as an external library, and this allowed us to visualise our core during development. Finally we have been using dynamically typed languages such as Python and JavaScript during our degree, and we wanted to each improve our programming skills with a statically typed language. Since C is devoid of object-oriented features like classes and methods, the workflow for our C library must follow a certain structure. The user of the C library must create **struct** instances and pass these to functions that alter them. However users of the included front-end need not worry about the implementation. Each of the functions, **structs**, and data types that are publicly available to the end user are forward declared in the file **at.h** which the user of our library can include in their project. This file includes all the function signatures, along with **struct** member descriptions to make it easy for the user to understand the data flow, as well as the use of our library. These forward declarations are fully defined in **at_internal**, or in their own respective **at_*.c** files, as to abstract the implementation away from the user.

The specification of the project involved the design of the following structures and data-types (each prefixed with **AT_** as part of our core library (Acoustic Tracer)):

The front-end, as mentioned is our method of creating a universal visualiser for the heat map of the model, while remaining platform agnostic (i.e. usable on MacOS, Linux, Windows, iOS, Android, etc.). Initially, during the project specification phase, we designed a communication standard which we stuck

with throughout the duration of the project, and it worked well. However, while the structure of the data remained the same, the data type had to change, which will be discussed later. We decided to create a server in C, that exposed an endpoint `/run` to the user, where they can send the scene and simulation configuration, and receive the heat map data as the server response. The front-end, could then make a HTTP request to the server which is easily performed on any language.

Simulation

Model

Creating the model was inherently the first step of our project, as that is the first step to the user flow. During our initial implementation specification design, we settled on the following model definition:

```
struct AT_Model {
    AT_Vec3 *vertices;
    AT_Vec3 *normals;
    uint32_t *indices;
    uint32_t *triangle_materials;
    size_t vertex_count;
    size_t index_count;
};
```

This creation of the `AT_Model` struct was possible with the library `cglTF`¹, which is a single-file/stb-style C glTF loader and writer. This library gives us the ability to parse the `.glb` file the user provides, resulting in access to the vertices, nodes, indices, and transformation matrices for each of the vertices (Rotation, Scale, and Quaternion). Initially the `AT_model_create()` function only extracted the raw vertex data, which worked initially. But as the need for more complex models arose, we had to alter our approach to make the use of `cglTF_node` attributes with the vertex data, combined with the parent and world transformation matrices, to give the vertex position and state in relation to the world.

Instead of naively extracting the raw vertex, index and normal data from the `.glb`, we must apply the parent and world rotation, scale, and quaternion matrices to each node. This is achieved by ‘walking’ up the node hierarchy, and multiplying each of the three matrices, until we arrive at the desired node. In this way, the three matrices accumulate across the parents, giving us an exact state of the node in relation to the ‘world’.

A model is a struct, composed of each of the following members:

- An array of vertices, each with an `x`, a `y`, and a `z` component, which represents a point in 3D space.
- An array of normals, which is the direction that the triangle ‘faces’. Represented as a three-dimensional vector.
- An array of indices, which outlines the order into which to draw lines from each vertex, creating the triangles of the model.
- An array of materials for each of the triangles, where the array is `index_count / 3` in length and the entry at `triangle_materials[t]` is the material for the triangle at `t`.
- A number representing the total amount of vertices in the model
- A number representing the total amount of indices in the model

Scene & Simulation

Creating the scene with `AT_scene_create()` is a simple aggregation of the `AT_SceneConfig` configuration, model (environment) pointer, and creation of the AABB (axis-aligned bounding box) of the model.

¹cglTF Library

Similarly the creation of the simulation with `AT_simulation_create()`, is another aggregation of the simulation settings (`AT_Settings`), calculation of the voxel grid dimensions, the allocation of the voxel grid, and the transfer of ownership of the `AT_Scene` pointer (similar to the scene transferring ownership of the model pointer).

Core Simulation Phases

Our core C library, as mentioned, involves the simulation of sound as rays throughout a three-dimensional environment. This implementation involves two main steps:

1. Ray Bounce Tree
2. DDA (Digital Differential Analyser) Voxel Sweep

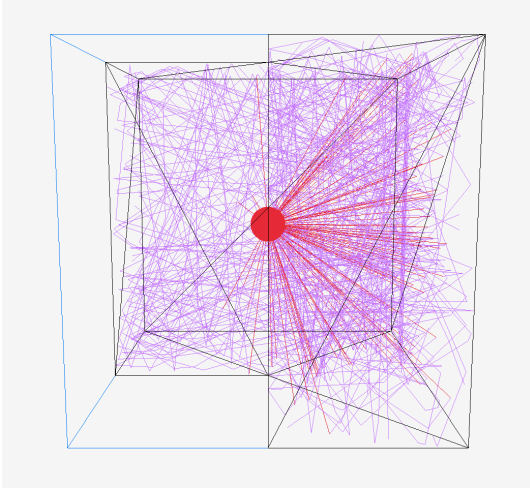


Figure 2: Phase 1 — Ray Bounce Tree

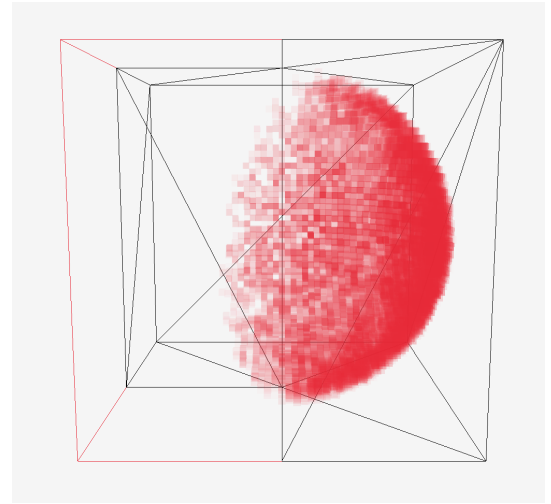


Figure 3: Phase 2 — DDA Voxel Sweep

Rays

A ray, for our purposes in this project, is a 3D representation of a line that extends infinitely from a source point (represented as a three-dimensional vector $\{x, y, z\}$), in a given direction (also represented as a three-dimensional vector $\{dx, dy, dz\}$, where each of the components is the change of the position along the x, y, and z axes, respectively).

Our first step in the implementation of the project was to emit rays from the ‘speaker’ (which will be hereby referred to as the ‘source’). Let’s assume for the moment that we have the scene configuration and the simulation configuration which are defined as follows:

```
typedef struct {
    const AT_Source *sources;
    uint32_t num_sources;
    AT_MaterialType material;
    const AT_Model *environment;
} AT_SceneConfig;

typedef struct {
    AT_Vec3 position;
    AT_Vec3 direction;
} AT_Source;
```

Where the `environment` is the `struct` of the 3D model the user specifies, and the source is composed of a position and direction (similar to the vector). We initialise rays using `AT_ray_init()` at the position of the source, setting their initial direction and position to those of the source they come from. Since we then want to create a realistic simulation of how these rays interact (bounce, scatter, and absorb) throughout the environment, we must implement each phenomena associated with these ‘sound rays’.

The first phenomena of the rays we implemented was reflection. This is where a ray ‘bounces’ off a surface, creating a new ray with an origin of the point of intersection of the surface, and a ‘reflected’ direction. Before we implemented reflection, we first had to implement the intersection between a ray and a triangle. Since our model is composed of entirely triangles, it is important to define what it means for a ray to intersect with a triangle.

This is where we used the Möller-Trumbore ray/triangle intersection algorithm². This allows us to compute whether a ray intersects with a triangle without having to pre-compute the plane equation of the plane containing the triangle.

Once we were able to get whether a ray intersected with a triangle, the next issue was to handle what actually happens when the ray intersects with the triangle. Initially we implemented a `hit_list` which was given to each ray, that tracked each point on any triangle that the ray intersected with. We must keep track of every single point of intersection, and then only ‘handle’ the point of intersection closest to the ray origin, as this would be the ‘first’ intersection, regardless of the order of the triangle checks. This caused problems with memory management, which is important in C. Since we didn’t know until runtime, how many times a ray would intersect with any of the triangles, we didn’t know how big to make the `hit_list` array. This introduced the concept of the dynamic array, which functions like lists do in Python and JavaScript, where the size of the list increases, if appending an element exceeds the capacity of the list. Once we had computed the closest point of intersection of each ray, we could then initialise a new ray with the origin of the point of intersection and the ‘reflected’ direction.

To combat the memory management associated with the `hit_list` approach of calculating the closest point of intersection, we pivoted to using a linked list approach for the ray implementation. Each ray would have a `child` ray that is the resultant ray after intersection, scattering, reflection etc. Upon the first intersection of a ray we would initialise a new ray with the computed origin and direction, and only update its direction and origin upon subsequent intersections, only if the distance from that point of intersection was less than the distance from the current rays origin to its child. This greatly simplified the computation, while also introducing an hierarchy among the rays, with a parent/child relationship. We could also easily navigate this ray hierarchy using linked list traversal methods Figure 2 shows a visualisation of the ray bounce tree in action, where each red line represents the `parent` rays and the purple lines represent all of the `child` rays.

Another ray phenomena implemented in our project is material absorption. Since this is not a concern of the end user of the library, this is defined in our `at_internal.h` header file where we declare functions, enumerations, and structs, to be used internally. For the absorption (and later scattering) we implemented, we decided to declare a table of materials along with their coefficients:

```
static const AT_Material AT_MATERIAL_TABLE[AT_MATERIAL_COUNT] = {
    [AT_MATERIAL_CONCRETE] = {.absorption = 0.02f, .scattering = 0.10f},
    [AT_MATERIAL_PLASTIC] = {.absorption = 0.03f, .scattering = 0.05f},
    [AT_MATERIAL_WOOD] = {.absorption = 0.10f, .scattering = 0.20f},
};
```

The scattering coefficient determines the probability that a ray, upon hitting a surface, is redirected diffusely in a random direction within the hemisphere above the surface normal, rather than following the

²Möller-Trumbore Intersection Algorithm

angle of perfect specular reflection. A rough concrete wall scatters more than a smooth plastic surface, which is reflected in the coefficients above.

Upon intersecting with a triangle, (whose material has been decided during `AT_scene_create()` as stated above), we can calculate the rays resultant energy modelled with the formula:

```
child->energy = ray->energy * (1.0f - triangle_material.absorption_coefficient);
```

This accurately uses the absorption coefficient for each material to alter the resultant energy of the `child` ray, created from the intersection.

The initial direction of each ray is also not arbitrary. Rather than emitting rays uniformly in all directions, we used cosine-weighted hemisphere sampling, implemented in `AT_sample_cosine_hemisphere()` in `at_utils.h`, with reference to the sampling method described in the PBR Book³. The function constructs an orthonormal basis from the source's direction vector, and samples a direction in the hemisphere above that surface using two uniform random floats. The polar angle is derived as `acos(sqrt(1 - u))` rather than `acos(u)`, and it is this square root that produces the cosine weighting as a consequence of the geometry of the sphere, biasing rays towards the forward direction of the source and away from grazing angles. The same function is also used when a ray scatters diffusely off a surface.

Voxels

The second phase of the simulation is the DDA voxel sweep. A voxel is a volumetric pixel, and in our case, a voxel is a dynamic array defined as follows:

```
typedef struct {
    float *items;
    size_t count;
    size_t capacity;
} AT_Voxel;
```

This structure allows us to use our general purpose dynamic array functionality defined in `at_utils.h`. `items` is a pointer to an array of floats, which we call “bins”. The concept of a bin is essential to the temporal aspect of the simulation. Each bin stores the total amount of sound energy that arrived in the voxel during a specific time interval. Think of each voxel as having a timeline, where each slot on the timeline records how much sound energy passes through the voxel within that window. During the replay, the bins are revealed sequentially.

In this phase, a voxel grid stores the spatial distribution of sound energy across the scene. The size of the voxels is user-specified, this size determines the resolution of the simulation. Smaller voxels produce a more detailed heat map but require significantly more memory and computation. The world dimensions of the voxel grid are calculated by subtracting the max and min value of the scenes AABB, which is the smallest possible box that encapsulates the entire model. These world dimensions are then used to calculate the grid dimensions by dividing the world dimensions by the user-specified voxel size, which gives `grid_x`, `grid_y` and `grid_z`.

Rather than allocating a 3-dimensional matrix, the voxel grid is stored in a 1-dimensional array of `AT_Voxel` types. A voxel at position (x, y, z) in the grid can be accessed using the formula `z * grid_y * grid_x + y * grid_x + x`. A contiguous block of memory is more efficient to allocate and iterate over than nested pointers.

During the planning phase of the project, our initial design for the voxel grid used a fixed number of bins per voxel, derived from a predetermined replay length:

³PBR Book

```
typedef struct {
    float energy_bins[NUM_BINS];
} Voxel_t
```

This approach required knowing the total simulation duration upfront in order to allocate the correct number of bins, though in practice this was not always possible. The total duration of a simulation depends on how far the rays travel before their energy drops below the termination threshold, which in turn depends on the geometry of the scene and the configuration of the source. The amount of times a ray bounces and how it loses energy throughout the environment is not something we can know until the simulation is run. We therefore moved to the dynamic array approach described above, where each voxel’s bin array grows dynamically as energy is deposited into new time frames.

Every ray generated in the first phase of the simulation is then traversed using the Digital Differential Analyzer (DDA) algorithm, implemented with reference to Amanatides and Woo’s *A Fast Voxel Traversal Algorithm for Ray Tracing* algorithm⁴. A naive approach to this problem might sample points along a ray at fixed intervals, but this risks skipping voxels entirely if they are only grazed by a ray, and also opens up the possibility for a voxel to be visited multiple times. The DDA algorithm allows us to track how far along a ray segment we need to travel to cross the next voxel boundary per axis, which is tracked by the variable `t_max`. At each step, the axis with the smallest `t_max` is advanced. This guarantees that every voxel the ray segment passes through is visited exactly once, regardless of the ray’s direction or the size of the voxels.

For each voxel the ray crosses, an amount of energy is deposited into that voxel. This energy deposit is weighted by three physical factors. First, the length of the ray segment inside the voxel, a ray travelling a longer path through a voxel contributes more energy to it. Second, inverse square attenuation with total distance `d` from the source, modelling how sound naturally loses intensity over distance. Third, air absorption, modelled as $\exp(-k * d)$, where `k` is the air coefficient, which accounts for energy lost to the medium itself as the ‘wave’ propagates. The result of these three factors combined is a single float value that is added to the voxel’s current bin.

The current time of the simulation `t` is calculated as `d / v` where `v` is the current simulation speed. Throughout the development of this project, `v` was one of two values. First being 343 m/s, which is the speed of sound, and an arbitrary slower speed that was used while building the visualisation, to make the movement of the sound energy through the heat map easier to observe. The current “bin” index is then calculated with `floor(t / bin_width)`, where `bin_width = 1 / fps`. This is the design decision that makes the output a temporal result rather than a static energy snapshot. Each voxel records how much energy arrived, as well as when it arrived. Each voxel’s bin array grows dynamically since at allocation time the duration of the simulation is not yet known. Figure 3 shows the resulting voxel heat map for the same scene, where the colour intensity of each voxel represents the amount of sound energy deposited into it across all time bins.

C Library

As mentioned before, C has no classes or namespaces. Building a library with a clean public interface that hides internals therefore requires deliberate design choices. The following describes the pattern we used to achieve this.

Public API (`at.h`)

The only file a user of our library needs to include is `at.h`. This file contains all of the function signatures, type definitions and enum declarations that are publicly available. The three main types of the library are

⁴Amanatides-Woo Voxel Traversal Algorithm

declared here as incomplete types:

```
typedef struct AT_Model AT_Model;
typedef struct AT_Scene AT_Scene;
typedef struct AT_Simulation AT_Simulation;
```

Because they are incomplete types, a user can hold a pointer to them but cannot dereference them to access their members directly. The full struct definitions are never visible outside the library. This also allows us to change the internal layout of any struct without breaking any code that uses the library, because the user only ever holds a pointer to an incomplete type, the compiler never needs to know the size of the struct itself.

All publicly available functions follow the same naming convention, prefixed with `AT_` and the name of the type they operate on:

```
AT_Result AT_model_create(AT_Model **out_model, const char *filepath);
void      AT_model_destroy(AT_Model *model);

AT_Result AT_scene_create(AT_Scene **out_scene, const AT_SceneConfig *config);
void      AT_scene_destroy(AT_Scene *scene);

AT_Result AT_simulation_create(AT_Simulation **out_simulation,
                              const AT_Scene *scene,
                              const AT_Settings *settings);
AT_Result AT_simulation_run(AT_Simulation *simulation);
void      AT_simulation_destroy(AT_Simulation *simulation);
```

The `AT_` prefix was a deliberate decision made during the planning phase of this project. Without it, names like `model_create` or `simulation_run` could easily clash with function names from other libraries, producing confusing linker errors. The prefix sort of acts like a manual namespace.

A typical usage of the library follows a consistent create, use, destroy cycle:

```
AT_Model *model = NULL;
if (AT_model_create(&model, "room.glb") != AT_OK) {
    fprintf(stderr, "Failed to load model\n");
    return 1;
}
```

```
AT_Source source = {
    .direction = {{0.0f, 1.0f, 0.0f}},
    .position = {{0.0f, 0.0f, 0.0f}}
};
```

```
AT_SceneConfig config = {
    .environment = model,
    .sources = &source,
    .num_sources = 1,
    .material = AT_MATERIAL_CONCRETE,
};
```

```
AT_Scene *scene = NULL;
if (AT_scene_create(&scene, &config) != AT_OK) {
```



```

    fprintf(stderr, "Failed to create scene\n");
    return 1;
}

AT_Settings settings = {
    .fps = 60,
    .num_rays = 1000,
    .voxel_size = 1.0f
};

AT_Simulation *sim = NULL;
if (AT_simulation_create(&sim, scene, &settings) != AT_OK) {
    fprintf(stderr, "Failed to create simulation\n");
    return 1;
}

AT_simulation_run(sim);

AT_simulation_destroy(sim);
AT_scene_destroy(scene);
AT_model_destroy(model);

```

The user must destroy in reverse of the creation order. This is because `AT_Simulation` borrows a pointer to `AT_Scene` and `AT_Scene` borrows a pointer to `AT_Model`. The ownership convention is documented well within the internal codebase.

Internal Architecture (`at_internal.h`)

The full struct definitions for all three opaque types live in `at_internal.h`, along with any other types that need to be shared across multiple source files but should never be exposed publicly. The reason that this internal header exists, as opposed to simply defining the structs within their respective `.c` files, is that multiple source files need access to full definitions simultaneously.

The full definitions are as follows:

```

struct AT_Scene {
    AT_Source *sources;
    AT_AABB world_AABB;
    uint32_t num_sources;
    AT_MaterialType material;
    const AT_Model *environment;
    //...
};

struct AT_Simulation {
    const AT_Scene *scene;
    AT_Voxel *voxel_grid;
    AT_Ray *rays;
    AT_Vec3 origin;
    AT_Vec3 dimensions;
    AT_Vec3 grid_dimensions;
    float voxel_size;
};

```

```

    float bin_width;
    uint32_t num_rays;
    uint32_t num_voxels;
    uint8_t fps;
};

```

The chain of ownership can be seen directly in the struct definitions. `AT_Simulation` holds a `const AT_Scene *` and `AT_Scene` holds a `const AT_Model *`. The `const` keyword here signals that these are borrowed references, and that the struct is not responsible for freeing them. In contrast, `AT_Voxel *voxel_grid` inside `AT_Simulation` carries no `const`, meaning the simulation owns that data outright, and `AT_simulation_destroy` is responsible for freeing it. This is also why `destroy` calls must be made in reverse order of creation. `AT_Simulation` must be freed before `AT_Scene`, because freeing the scene while the simulation still holds a pointer to it would leave the simulation with a dangling reference.

Error Handling (AT_Result)

Every function in the library that could potentially fail returns an `AT_Result`:

```

typedef enum {
    AT_OK = 0,
    AT_ERR_INVALID_ARGUMENT,
    AT_ERR_ALLOC_ERROR,
    AT_ERR_NETWORK_FAILURE
} AT_Result;

```

To combat the fact that C can't throw exceptions, the convention is to return an error code that the caller must explicitly check. This is particularly important for our library since the simulation involves a large number of heap allocations, any of which could fail. Detecting these failures early produces a clear error message, rather than a segmentation fault.

`at_result.h` also provides a small helper function `AT_handle_result()` which prints the error type and a custom message to `stderr`, used throughout development to quickly surface allocation and argument errors without having to write a switch case every time.

Communication between the Core and the Front-End

With our exemplar front-end web application, the workflow consists of the front-end sending the following data to the core back-end:

```

{
  "filepath": "<path_to_glb>",
  "voxel_size": "<size_of_voxel {float}>",
  "material": "<model_material>",
  "fps": "<fps {uint8}>",
  "num_rays": "<num_rays {uint32}>",
  "source": {
    "position": [<pos_x {uint32}>, "<pos_y {uint32}>", "<pos_z {uint32}>"],
    "direction": [<dir_x {uint32}>, "<dir_y {uint32}>", "<dir_z {uint32}>"]
  }
}

```

Since this JSON is quite minimal, it is acceptable to send as-is.

The back-end for our application is centred around a C server whose implementation is defined in

`backend/at_net.c`. This involves setting up a TCP socket (which essentially behaves like a HTTP server). It listens for incoming connections, accepts only `POST /run` requests, parses the incoming configuration and settings, runs the ray-tracer, converts the frame data (voxel heat-map) to binary, and finally writes the binary stream as the response. The front-end receives this stream as a response, and can then parse the result, and construct the visualisation of the heat-map on the client-side.

During our initial design phase, we outlined the communication standard for the voxel data as shown:

```
{
  "1": [{"1": 24}, {"3": 34}, {"6": 32}, {"13": 45}, ...]
  "2": [...]
  .
  .
  .
  "n": [...]
}
```

Here, the first key is the frame number, and the value is a list of **key: value** pairs where the key is the voxel index, and the value is the energy of the voxel at that current frame. We found that this was an easy to understand communication standard, while reducing the amount of total tokens required to transmit. Furthermore it is natively JSON, so it is easy to parse on the front-end application. In the final iteration we prefixed the frame number with `frame_` to distinguish between the frame number and the voxel index.

On the back-end, we used the `cJSON` library ⁵, to parse the configuration received from the front-end.

Frontend

Introduction

Note: The project’s purpose, the C simulation engine, the binary communication standard, the public API (`at.h`), and the overall architecture are described in the preceding sections of this report. This section focuses exclusively on the browser frontend: how it configures, stores, and visualises simulations.

The AcousticTracer project, as described, pairs a C simulation engine (the core) with a browser-based frontend, which configures, stores, and visualises the simulation. Tools like ODEON exist for this domain, but they are desktop-only, expensive, and closed-source. A browser-based alternative would be freely accessible, require no installation, and provide immediate real-time use. Although the team member’s role on this project was frontend engineer, the nature of the work bore little resemblance to conventional frontend development. The typical concerns of a UI/UX-focused role were secondary throughout. The primary challenges were technical: writing per-frame transform matrices directly into GPU-backed buffers, normalising direction vectors with quaternion math, parsing the custom ATRB binary protocol described earlier, clamping 3D coordinates to axis-aligned bounding boxes during drag interactions, and orchestrating an asynchronous pipeline spanning two independent backends.

This focus was not a deliberate de-prioritisation of design, but an accurate reflection of where the complexity lay. A voxel renderer that drops frames is unusable regardless of how polished its surrounding UI is. A binary parser that misaligns a single byte offset produces garbage data that no amount of styling can mask. The frontend’s value to the project was measured not in visual refinement but in its ability to bridge the gap between a C simulation engine and a browser-based 3D visualisation.

⁵`cJSON` library

Core Goals

The frontend has three responsibilities that we discussed and outlined early in development but did not yet know how to tackle, and each came with distinct technical demands:

1. **Configure and submit** — The user uploads a `.glb` room model, sets simulation parameters (voxel size, ray count, FPS, surface material), and interactively places a sound source by adjusting its position inside the 3D scene. The configuration is sent to the C backend as JSON matching the communication standard defined earlier.
2. **Decode and store** — The C backend returns results in the ATRB binary format. The frontend decodes this into typed arrays, uploads it to Appwrite file storage, and caches the parsed result so revisiting a completed simulation is instantaneous.
3. **Render and replay** — The decoded frames are visualised as a 3D voxel heat map overlaid on the room model. A typical simulation at `voxelSize = 0.05` on a $5\text{m} \times 3\text{m} \times 4\text{m}$ room produces 480,000 voxels, each requiring per-frame position and colour updates at interactive frame rates.

Initial Challenges

The design was shaped by three challenges that recur throughout this section. First, the performance budget: hundreds of thousands of voxels multiplied by 60 FPS and per-instance matrix and colour updates eliminated most naive rendering approaches. Second, the complexity of 3D software development in a browser, which required working with concepts that have no equivalent in conventional web development: scene graphs, projection matrices, quaternion rotations, raycasting for hit detection, and GPU-instanced rendering. Third, the coupling between backends: the frontend mediates between Appwrite (authentication, database, file storage) and the C ray-tracer (custom HTTP endpoint), and the data layer must sit between both without leaking SDK details into components.

A Learning Journey Through the React Ecosystem

This project was the frontend developer's first experience building a fullstack web application within the React ecosystem, and with 3D Web rendering thrown into the mix it was quite a challenge to undertake. Every major technology in the stack (React itself, TypeScript, Zustand, TanStack Query, Three.js, React Three Fiber, Tailwind CSS, Appwrite) was either encountered for the first time during development or a technology the frontend developer had limited experience with. The consequence of this is visible in the project's commit history, which records not just feature additions but a series of architectural refactors, each driven by the discovery of a better approach to a problem that needs to be solved or had already been solved in a less effective way.

The pattern of implementation was quite consistent throughout the development of the project. It would start with an attempt to implement a feature with the tools and patterns understood at the time, encounter the limitations of that approach under real conditions during testing, discover a more appropriate technology or pattern, and refactor. These refactors were uncomfortable but correct — each one required understanding *why* the previous approach failed, not just *that* it failed, and this understanding informed every subsequent decision.

Technology Decisions

Before delving into the architectural design and implementation, this subsection briefly describes the technologies used and why they were chosen.

React React was chosen as the UI framework because the frontend developer had invested time developing skills with it before and during the early stages of the project, completing Scrimba’s introductory course and the majority of the advanced React courses, as well as their TypeScript course. That preparation provided enough fluency with React’s component model, hooks, and ecosystem to be productive from the first week.

JavaScript -> TypeScript The initial scaffold was plain JavaScript. It quickly became apparent that JavaScript alone would not be sufficient, the project was hitting runtime crashes caused by misspelled prop names and `undefined` values propagating silently through the component tree, bugs that TypeScript’s structural type system catches at compile time. The frontend developer invested time during the first three weeks of development learning TypeScript alongside developing the codebase.

CSS → Tailwind CSS

Hand-written CSS files worked fine while the project had five components. Once that count reached fifteen, issues started to arise. Tailwind eliminated this problem entirely by moving styling into utility classes located within the JSX. This co-location is where Tailwind and React complement each other naturally, because React components are self-contained, having the styling live inline as class names means a component’s appearance, behavior, and structure are all visible in a single file. Tailwind v4’s CSS-native `@theme` directives allowed us to define project-wide design tokens directly in `index.css`.

```
@import "tailwindcss";

@theme {
  --color-bg-primary: #2d2d39;
  --color-bg-card: #282833;
  --color-text-primary: #ffffff;
  --color-accent: #fbbf24;
  --color-button-primary: #4f46e5;
  --color-danger: #f87171;
  --color-success: #34d399;
}
```

useState -> Zustand -> Zustand + TanStack Query

State management went through three distinct phases, each driven by the limitations of the previous approach. This evolution is discussed in detail in the State Management section below.

Late-stage UI design

Towards the end of the project, after the core features of the frontend had been developed and optimised, the need for a polished UI could no longer be ignored. Dropdown menus with keyboard navigation, accessible dialogs, and responsive tables were all needed, and building these from scratch would have been a significant time investment that we did not have the luxury of. We found **shadcn/ui**, a collection of copy-and-paste React components built on **Radix UI** primitives and styled with Tailwind. Rather than installing a monolithic library, shadcn/ui provides individual component files that we own and modify. This approach aligned with our existing Tailwind-based styling and allowed incremental adoption. If we were to start this project again, we would have leveraged shadcn far earlier in the development process.

Architectural Overview

Feature Orientated Architecture

Roughly two weeks into development, the codebase was restructured from a flat component directory into the **Bulletproof React** pattern, a feature orientated architecture where each major feature is a self-contained directory:

```
web/src/
+-- app/           # Shell: provider composition, router, global CSS
+-- api/           # Barrel re-exports the data contracts
+-- components/    # Shared UI
+-- features/
|   +-- auth/      # Login, Register, Settings, OAuth, UserProvider
|   +-- simulation/ # Everything acoustic: API, components, hooks, routes, store
+-- lib/           # Infrastructure: Appwrite client, QueryClient, utils
+-- utils/         # Pure helpers
```

This codebase structure enforces the concept of **import direction**: feature code may import from `lib/` and `components/`, but never from another feature directly. The `auth` and `simulation` features communicate only through the provider hierarchy and through barrel exports in `api/`. This structure was well thought out and easy to navigate and understand once it was explained to the rest of the team. Similar to our C library's use of `at.h` as the single public interface, each feature exposes its functionality through a controlled set of barrel exports, keeping the internals hidden from the rest of the application.

Provider Hierarchy

In React, a **provider** is a component that makes shared data or services available to every component nested inside it, without having to pass that data down manually through props at each level of the component tree. Providers were used to compose the application's global infrastructure (error handling, loading states, data caching, authentication) into a single wrapper so that every page and component has access to these services automatically.

The application's provider stack is composed in `provider.tsx`:

```
<ErrorBoundary FallbackComponent={MainErrorFallback}>
  <Suspense fallback={<LoadingSpinner />}>
    <QueryClientProvider client={queryClient}>
      <UserProvider>
        {children}
      <ToastContainer />
    </UserProvider>
  </QueryClientProvider>
</Suspense>
</ErrorBoundary>
```

The ordering is incredibly important and deliberate:

1. **ErrorBoundary** - catches any uncaught exception, including `Suspense` promise rejections, and renders a recovery UI. Nothing can escape this boundary.
2. **Suspense** - displays a loading spinner while any descendant component suspends (e.g., during lazy-loaded route fetching).
3. **QueryClientProvider** - makes the TanStack Query cache available to all descendants.

4. **UserProvider** - initialises authentication state. On login and logout, it resets the Zustand SceneStore and clears the TanStack Query cache to prevent data leakage between sessions.

Core Abstractions

Before examining code, it is worth naming the five abstractions that the entire frontend is built around. Every component, hook, and data flow in the application is connected to one or more of these:

Abstraction	Responsibility	Implementation
Simulation	The domain entity: a configured acoustic experiment with its results. Exists in two shapes, a SimulationDocument (Appwrite's <code>snake_case</code> database row) and a Simulation (the camelCase object the web app uses).	<code>simulation-repository.ts</code>
SceneStore	The client-side state container for everything the 3D scene needs: model bounds, voxel size, selected source position/direction, UI toggles (wireframe, grid visibility), the pending upload file, and the current playback frame index.	Zustand store in <code>scene-store.ts</code>
SceneCanvas	The rendering surface. A React Three Fiber <code><Canvas></code> that bridges React's component model to Three.js's scene. Manages camera, lighting, model loading, and child component composition.	<code>scene-viewer.tsx</code>
VoxelGrid	The GPU-backed instanced voxel renderer. Given a bounding box, a voxel size, and an optional sequence of sparse energy frames, it maintains a single InstancedMesh with direct buffer writes for position and colour.	<code>voxel-grid.tsx</code>
Simulation Repository	The data access layer. Encapsulates every Appwrite SDK call behind typed methods (<code>list</code> , <code>getById</code> , <code>create</code> , <code>update</code> , <code>delete</code> , <code>uploadFile</code> , <code>getFileName</code>). No component ever imports Appwrite directly.	<code>simulation-repository.ts</code>

- Design choices (front)
 - state storing
 - storing (not state)
 - rendering
 - replaying
 - auth
 - routing
- What we would do different (ODEON)
- Lessons learned
- Future Feature Plans
- Conclusions
- references

C

- Implementation
 - Two phase simulation:
 - * Rays
 - Moller-Trumbmore alg, linked list/tree structure of rays, absorption...
 - * Voxels
 - DDA, energy bins design, voxel grid, attenuation...
- Design choices (c library)
 - at.h, opaque API, at_internal.h, returning AT_Result...
- Optimisation
- API comms (both, potent split into two sections)
 - bins

References